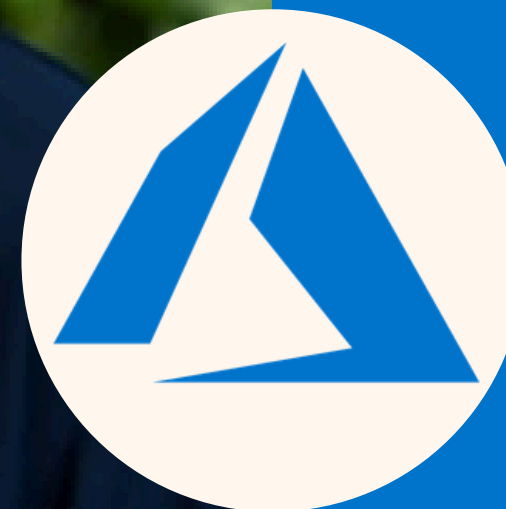


 **GET STARTED TODAY**

LUKE

GINN



AZURE
CLOUD
MASTERY



Unit 15 Introduction – Measuring Agent Quality

AZURE
CLOUD
MASTERY

Unit 14 covered Foundry Trace for debugging. Unit 15 covers performance evaluation – measuring how well your agent performs on three key metrics.

- **Review of Unit 14:** You learned to find failures with traces. Now you need to measure overall agent quality: does it call the right tools and follow instructions?
- **What This Unit Covers:** You will learn Tool Call Accuracy (did the agent call the correct API?), Task Adherence (did it follow system instructions?), and Intent Resolution (did it understand the user?).
- **Why Metrics Matter:** Without measurement, you cannot improve. The exam tests your ability to calculate and interpret these three metrics.

Exam Takeaway: Performance Evaluation is part of Observability domain (20% weighting). The exam tests Tool Call Accuracy, Task Adherence, and Intent Resolution.



Tool Call Accuracy – Did the Agent Call the Right Tool?

Tool Call Accuracy measures the percentage of tool calls where the agent selected the correct tool name and provided valid parameters for that tool.

- **Definition:** For each user request that requires a tool, did the agent call the right tool? Example: User asks "What is the weather?" Correct tool = ``get_weather``, not ``send_email``.
- **Correct Parameters Matter:** Even with the right tool name, parameters must be valid. ``get_weather(city="Seattle")`` is correct. ``get_weather(city=123)`` is incorrect.
- **Why It Matters:** Low accuracy means the agent does not understand which tool matches which intent. Users get wrong answers or error messages.

Exam Takeaway: Tool Call Accuracy = (number of correct tool calls) / (total tool calls).
The exam tests calculating accuracy from trace logs.



Calculating Tool Call Accuracy

To calculate tool call accuracy, review a sample of agent conversations, count correct vs. incorrect tool calls, and divide correct by total.

- **Step 1 – Collect Sample:** Select 100 conversations where the agent made tool calls. Exclude conversations with no tool calls.
- **Step 2 – Label Each Call:** For each tool call, label as Correct (right tool + valid parameters) or Incorrect (wrong tool or invalid parameters).
- **Step 3 – Calculate:** $\text{Accuracy} = (\text{Correct Calls}) / (\text{Total Calls})$. Example: 85 correct out of 100 total = 85% accuracy.
- **Threshold for Production:** Target >95% accuracy. Below 90% means the agent needs better tool descriptions or system instructions.

Exam Takeaway: The exam tests calculation and interpretation: $85/100 = 85\%$ accuracy, which is below production threshold (needs improvement).



Low tool call accuracy usually results from poor tool descriptions, ambiguous tool names, or missing parameter examples in the tool definition.

- **Poor Description:** Tool description is "Gets data" instead of "Gets current weather for a city by name." Agent cannot distinguish between weather and stock tools.
- **Ambiguous Names:** ``get_info`` vs. ``get_data`` – agent cannot tell which tool does what. Names must be specific: ``get_weather``, ``get_stock_price``.
- **Missing Parameter Examples:** Tool definition does not show example values. Agent invents parameters like ``city=123`` instead of ``city="Seattle"``.

Exam Takeaway: The exam presents low accuracy scenarios. You must identify the root cause: ambiguous names, poor descriptions, or missing examples.



Task Adherence – Did the Agent Follow Instructions?

Task Adherence measures the percentage of conversations where the agent followed all system instructions and stayed within its defined boundaries.

- **Definition:** Task Adherence checks if the agent violated any rule in its system message. Example: System says "Never share internal prices." Agent shares a price = violation.
- **Measuring Adherence:** Review conversation samples. Mark each conversation as Adherent (no violations) or Non-Adherent (one or more violations).
- **Why It Matters:** Non-adherent agents may share sensitive data, refuse valid requests incorrectly, or act outside their approved scope.

Exam Takeaway: Task Adherence = (Adherent Conversations) / (Total Conversations). The exam tests identifying violations from conversation transcripts.



Types of Task Adherence Violations

Task adherence violations fall into three categories: boundary violations (did something prohibited), instruction skipping (ignored a required step), or persona drift (changed tone/role).

- **Boundary Violation:** System says "Never delete data." Agent deletes a record. Most severe violation. Requires immediate fix.
- **Instruction Skipping:** System says "Always ask for order number before looking up account." Agent looks up account without asking. Less severe but still a violation.
- **Persona Drift:** System says "You are a professional support agent." Agent uses slang or makes jokes. Violation of tone rules.

Exam Takeaway: The exam presents conversation examples and asks which type of task adherence violation occurred (boundary, skipping, or drift).



Intent Resolution – Did the Agent Understand the User?

AZURE CLOUD MASTERY

Intent Resolution measures the percentage of user requests where the agent correctly understood what the user wanted, regardless of whether it could fulfill it.

- **Definition:** Intent Resolution checks if the agent identified the correct user goal.
Example: User says "My order hasn't arrived." Correct intent = "check order status."
- **Correct Intent, Failed Execution:** If agent identifies "check order status" but the tool fails, Intent Resolution is still correct. Execution is separate.
- **Why It Matters:** If the agent misunderstands intent, it will call wrong tools or give irrelevant answers, frustrating users.

Exam Takeaway: $\text{Intent Resolution} = (\text{Correctly Identified Intents}) / (\text{Total User Requests})$.
The exam tests distinguishing intent from execution success.



To measure intent resolution, create a ground truth dataset where humans label the correct intent for each user request, then compare agent predictions.

- **Step 1 – Create Ground Truth:** Sample 500 user requests. Have two human labelers independently assign an intent category (e.g., "refund_request", "order_status", "product_question").
- **Step 2 – Resolve Disagreements:** Where labelers disagree, a third labeler decides. This creates the "correct" intent for each request.
- **Step 3 – Compare Agent Output:** Run each request through the agent. Record the intent the agent inferred (from its tool call or response).
- **Step 4 – Calculate:** Resolution Rate = (Agent Intent Matches Ground Truth) / (Total Requests).

Exam Takeaway: Ground truth requires human labeling. The exam tests understanding of the measurement process, not just the formula.



Intent Resolution Failure Patterns

Intent resolution failures fall into three patterns: wrong category (misunderstood domain), too vague (could not choose), or hallucinated (invented non-existent intent).

- **Wrong Category:** User asks "Cancel my order" (intent: cancellation). Agent infers "order status" (wrong category). Completely different action.
- **Too Vague:** Agent returns "I'm not sure what you want" or asks for clarification when intent was clear. Indicates weak system instructions.
- **Hallucinated Intent:** User asks "What is your name?" Agent infers "refund_request" – inventing an intent that does not exist in the user's request.

Exam Takeaway: The exam presents intent failure examples. You must identify which pattern (wrong category, vague, or hallucinated) occurred.



Tool Call Accuracy vs. Intent Resolution

AZURE
CLOUD
MASTERY

Tool call accuracy and intent resolution measure different things: intent resolution checks understanding; tool call accuracy checks execution of that understanding.

- **High Intent, Low Tool Accuracy:** Agent understands "get weather" but calls wrong weather API or uses wrong city format. Fix tool definitions.
- **Low Intent, High Tool Accuracy:** Agent misunderstands "cancel order" as "order status" but correctly calls order status tool. Fix system instructions or model.
- **Both Low:** Agent has fundamental problems. Upgrade to a more capable model (GPT-4o) and rewrite system instructions from scratch.

Exam Takeaway: The exam presents paired metrics and asks which component to fix – system instructions (intent) or tool definitions (accuracy).



Task Adherence vs. Intent Resolution

Task adherence and intent resolution are independent: an agent can correctly understand intent (good resolution) but still violate system instructions (bad adherence).

- **Example – Good Resolution, Bad Adherence:** User asks "Delete my account." Agent correctly identifies intent (deletion). But system says "Never delete accounts without approval." Agent deletes anyway = violation.
- **Example – Bad Resolution, Good Adherence:** User asks "Cancel my order." Agent thinks user wants "refund status." Agent correctly follows system instruction for refund status (good adherence) but wrong intent.
- **Prioritization:** Fix adherence first (security risk), then resolution (user experience).

Exam Takeaway: Adherence violations are security risks and must be fixed before resolution. The exam tests prioritization between metrics.



Measuring Performance with Foundry Trace

AZURE
CLOUD
MASTERY

Foundry Trace can automatically calculate tool call accuracy and task adherence by analyzing span attributes and system instruction boundaries.

- **Tool Call Accuracy from Traces:** Query traces for ``span.name == "tool_call"``. Compare ``tool.name`` and ``parameters`` against expected values from ground truth.
- **Task Adherence from Traces:** Query traces for ``span.name == "llm_call"``. Scan response for content that violates system instruction keywords (e.g., "price" if system says "never share prices").
- **Intent Resolution from Tool Chains:** If agent calls ``get_weather`` for weather request, intent is correct. If calls ``get_stock_price`` for weather request, intent is wrong.

Exam Takeaway: The exam tests using Foundry Trace attributes to calculate metrics automatically, not just manually.



Performance Baselines and Targets

Set baseline metrics during development, then track improvement after each change.
Production targets vary by use case but have minimum thresholds.

- **Minimum Production Targets:** Tool Call Accuracy >95%, Task Adherence >99%, Intent Resolution >90%. Below these, agent is not ready for users.
- **Baseline Measurement:** Run 500 test conversations before any optimization. Record metrics as baseline (e.g., accuracy 70%, adherence 80%, resolution 65%).
- **Improvement Tracking:** After each change (better tool descriptions, stronger system instructions), rerun the same 500 tests. Compare to baseline.
- **Regression Detection:** If a change improves accuracy but hurts adherence, decide which metric is more important for your use case.

Exam Takeaway: The exam tests setting baselines, tracking improvement, and detecting regressions across metric changes.



Continuous Evaluation in CI/CD

Integrate performance evaluation into your CI/CD pipeline: run metric tests automatically before deploying agent changes to production.

- **Pre-Deployment Gate:** Before deploying to production, run 500 test conversations. If any metric falls below threshold (e.g., accuracy <95%), block deployment.
- **A/B Testing in Production:** Deploy two agent versions (current vs. candidate) to 10% of users each. Compare metrics after 1,000 conversations.
- **Alerting on Regressions:** Set up Azure Monitor alerts. If Tool Call Accuracy drops 5% in 1 hour, send alert to on-call engineer.
- **Automated Rollback:** If alert triggers, automatically redeploy the previous known-good version.

Exam Takeaway: The exam tests CI/CD integration: pre-deployment gates, A/B testing, alerting thresholds, and automated rollback.

Improving Low-Performing Metrics

Each low metric has specific fixes: low tool call accuracy → improve tool descriptions; low task adherence → strengthen system instructions; low intent resolution → upgrade model.

- **Fix Low Tool Call Accuracy:** Add examples to tool descriptions. Rename ambiguous tools. Test with 100 requests, verify improvement.
- **Fix Low Task Adherence:** Add explicit refusal examples to system instructions. Use red teaming to find boundary gaps. Add more hard boundary rules.
- **Fix Low Intent Resolution:** Upgrade from SLM (Phi-3) to LLM (GPT-4o) for intent classification. Add chain-of-thought instructions. Expand ground truth dataset.

Exam Takeaway: The exam tests matching low metrics to appropriate fixes – tool descriptions for accuracy, system instructions for adherence, better models for resolution.



Unit 15 Summary – Performance Evaluation

You have learned to measure three key agent metrics: Tool Call Accuracy, Task Adherence, and Intent Resolution, then use them to drive improvements.

- **Three Metrics:** Tool Call Accuracy (correct tool + parameters), Task Adherence (followed system instructions), Intent Resolution (understood user goal).
- **Measurement:** Use ground truth datasets, Foundry Trace attributes, and conversation sampling. Calculate rates from correct/total.
- **Improvement Strategies:** Better tool descriptions (accuracy), stronger system instructions (adherence), better models (resolution). Continuous evaluation in CI/CD.
- **Next Unit Preview:** Unit 16 introduces Agent Memory – short-term session context and long-term user preferences with Cosmos DB and Redis.

Exam Takeaway: Performance Evaluation is 20% of the exam. Master the three metrics, their calculation, root cause analysis, and improvement strategies.

**THANKS
FOR
WATCHING**

